

DESIGN AND ANALYSIS OF ALGORITHMS
LAB WORKBOOK WEEK – 8

NAME: MADDU RAAHITHYA YADAV
ROLL NUMBER: CH.SC.U4CSE24124
CLASS: CSE-B

Write a C program to implement Huffman Coding for the word "DATA ANALYTICS AND INTELLIGENCE LABORATORY". Generate and display the Huffman code for each character and Calculate and display the Average Code Length.

Working:

Construct the Huffman coding for the word:

["DATA ANALYTICS AND INTELLIGENCE LABORATORY"]

- Construct the Huffman Tree
- Assign binary codes to each character.
- Calculate the average code length

D-2, A-7, T-4, N-4, L-4, Y-2, I-3, C-2, S-1, E-3, G-1, B-1, O-2, R-2

Huffman Coding:-

Step 1:- Create nodes for frequency and arrange in ascending order.

① ① ① ② ② ② ② ② ③ ③ ④ ④ ④ ⑦

S G B D Y C O R I E T N L A

Arrange it in increasing order of Alphabets if we have repeated frequency

① ① ① ② ② ② ② ② ③ ③ ④ ④ ④ ⑦

B G S C D O R Y E I L N T A

Step 2:- Combine lowest frequency nodes into a third node ..

New frequency = sum of both nodes. and make the two nodes as left and right children.

② ① ② ② ② ② ② ③ ③ ④ ④ ④ ⑦

B G S C D O R Y E I L N T A

The newly created node is inserted back following the ascending order. so after rearranging.

→ ① ② ② ② ② ② ② ③ ③ ④ ④ ④ ⑦

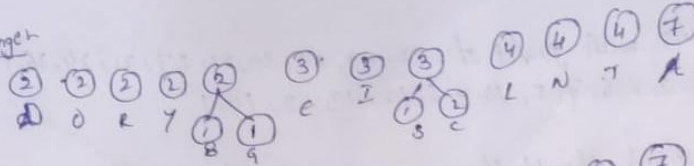
S C D O R Y B G E I L N T A

Step 3:-

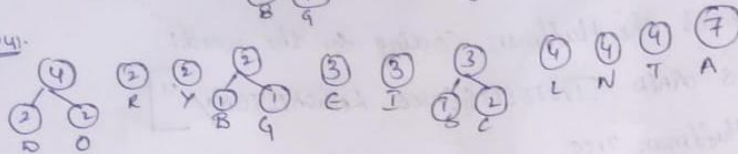
③ ② ② ② ② ② ③ ③ ④ ④ ④ ⑦

D O R Y B G E I L N T A

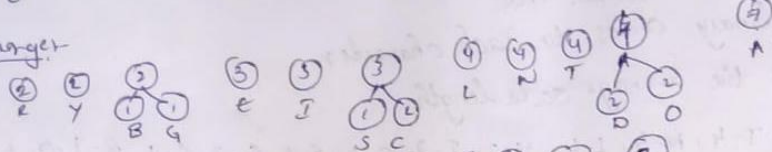
Rearrange



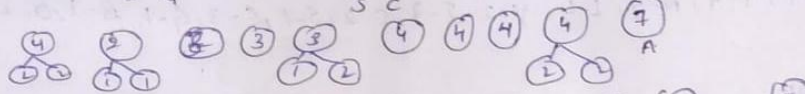
Step 4:-



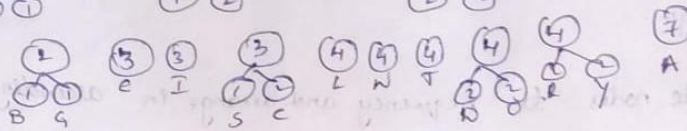
Rearrange



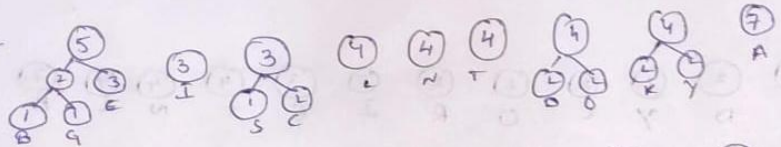
Step 1:-



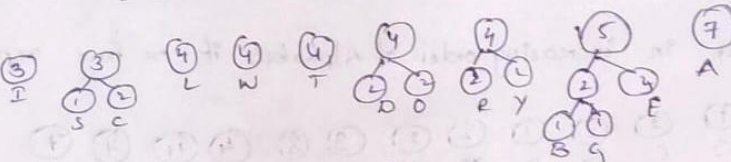
Rearrange



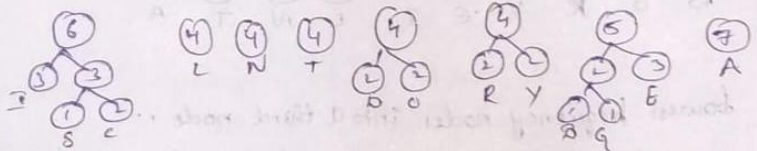
Step 6:-



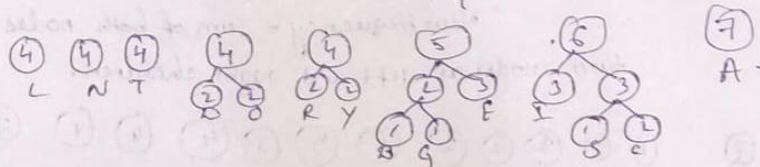
Rearrange



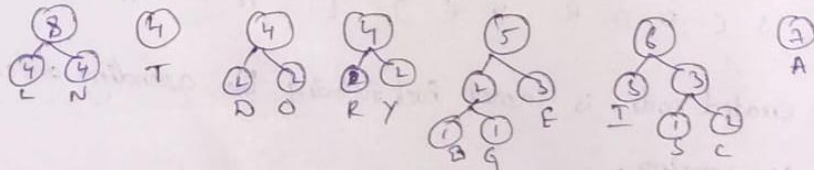
Step 7:-



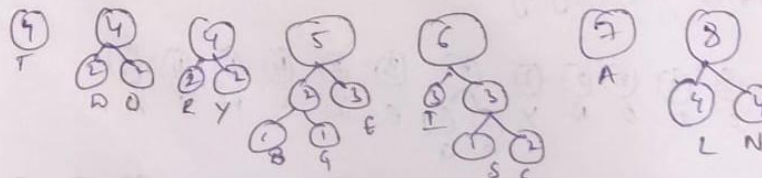
Rearrange 1:-



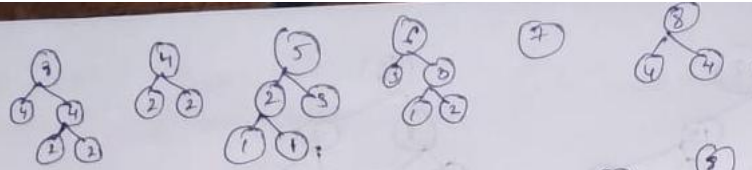
Step 8:-



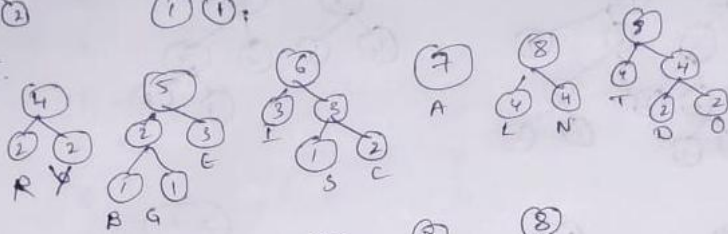
Rearrange



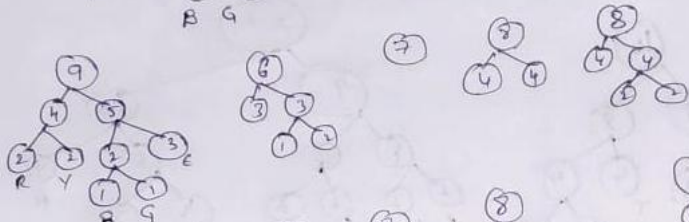
Step 9:-



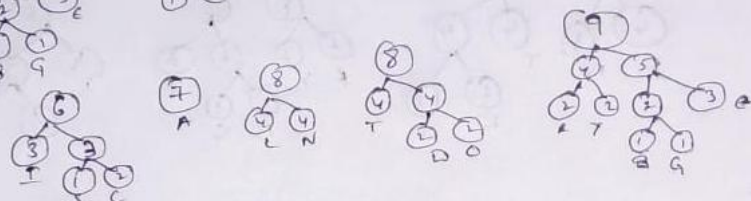
Rearrange:-



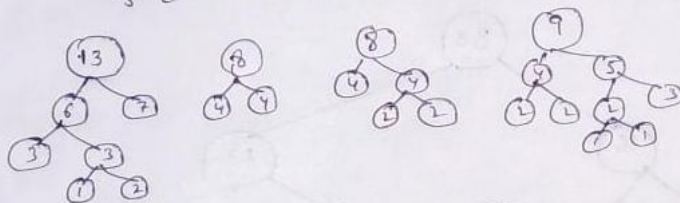
Step 10:-



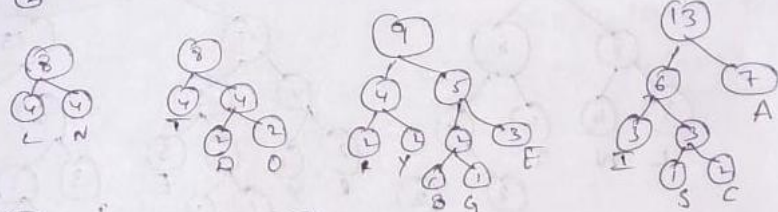
Rearrange:-



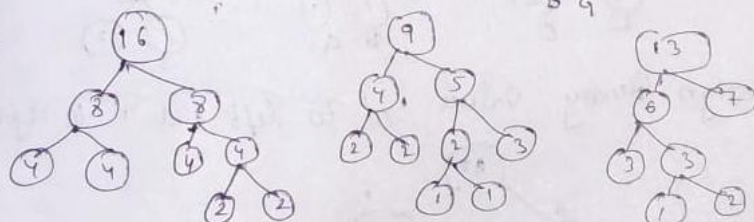
Step 11:-



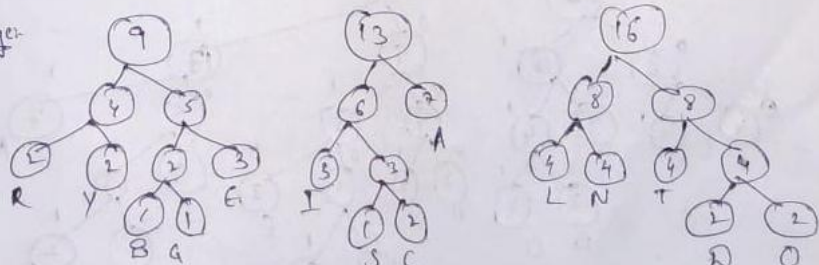
Rearrange:-



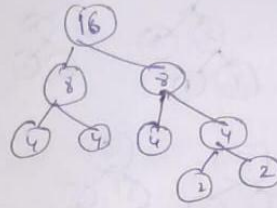
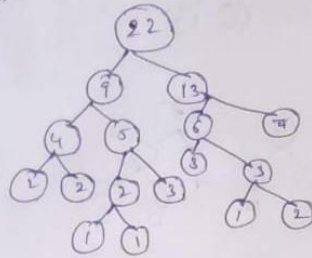
Step 12:-



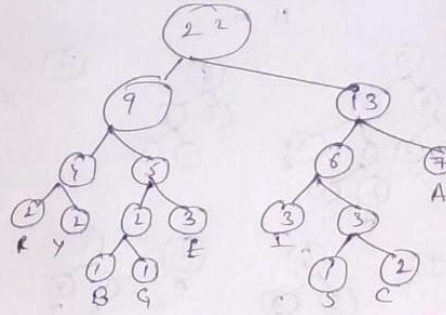
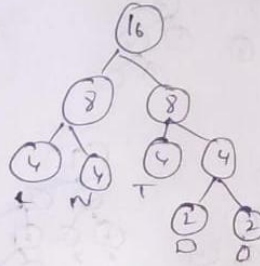
Rearrange:-



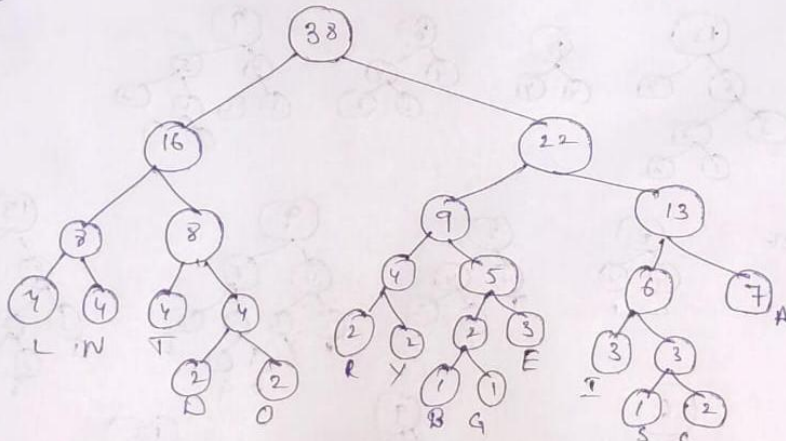
Step 13:-



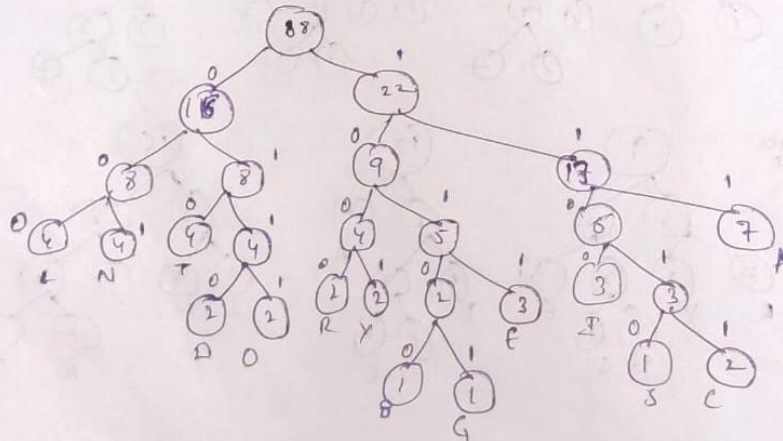
Rearrange:-



Step 14:-



Step 15:- Assign Binary Values '0' to left and '1' to right



<u>Step 16:-</u> Final Huffman Codes.		<u>length of code</u>
B	- 10100	5
G	- 10101	5
S	- 11010	5
C	- 11011	5
D	- 0110	4
O	- 0111	4
R	- 1000	4
Y	- 1001	4
E	- 1011	4
I	- 1100	4
L	- 000	3
N	- 001	3
T	- 010	3
A	- 111	3

$$\text{Average code length} \rightarrow \frac{\sum (f_i \times l_i)}{\sum f_i}$$

$$= \frac{(1 \times 5) + (1 \times 5) + (1 \times 5) + (2 \times 5) + 2(4) + 2(4) + 2(4) + 2(4) + (3 \times 4) + 3(4) + (4 \times 3) + 2(3) + (4 \times 3) + (7 \times 3)}{1+1+1+2+2+2+2+2+3+3+4+4+4+7}$$

$$= \frac{138}{38} = 3.631$$

$$\text{Length of Huffman encoded msg (in bits)} = \text{total length of character in msg} \times \text{Average code length}$$

$$= 38 \times 3.631$$

$$= 137.978$$

$$\approx 138 \text{ bits}$$

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #define MAX 100
5  struct Node {
6      char data;
7      int freq;
8      struct Node *left, *right;
9  };
10 struct Node* createNode(char data, int freq) {
11     struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
12     temp->data = data;
13     temp->freq = freq;
14     temp->left = temp->right = NULL;
15     return temp;
16 }
17 void swap(struct Node** a, struct Node** b) {
18     struct Node* t = *a;
19     *a = *b;
20     *b = t;
21 }
22 void sort(struct Node* arr[], int n) {
23     for(int i = 0; i < n-1; i++) {
24         for(int j = 0; j < n-i-1; j++) {
25             if(arr[j]->freq > arr[j+1]->freq) {
26                 swap(&arr[j], &arr[j+1]);
27             }
28         }
29     }
30 }
31 void printCodes(struct Node* root, int code[], int top, int *weightedSum) {
32     if(root->left) {
33         code[top] = 0;
34         printCodes(root->left, code, top + 1, weightedSum);
35     }
36     if(root->right) {
37         code[top] = 1;
38         printCodes(root->right, code, top + 1, weightedSum);
39     }
40     if(root->left == NULL && root->right == NULL) {
41         printf("%c : ", root->data);
```

```

42         for(int i = 0; i < top; i++)
43             printf("%d", code[i]);
44
45         printf(" (Length = %d)\n", top);
46         *weightedSum += root->freq * top;
47     }
48 }
49 int main() {
50     char input[200];
51     int freq[256] = {0};
52     struct Node* nodes[256];
53     int count = 0;
54     int totalFreq = 0;
55     printf("Enter a sentence: ");
56     fgets(input, sizeof(input), stdin);
57     int len = strlen(input);
58     for(int i = 0; i < len; i++) {
59         if(input[i] != ' ' && input[i] != '\n') {
60             freq[(int)input[i]]++;
61             totalFreq++;
62         }
63     }
64     for(int i = 0; i < 256; i++) {
65         if(freq[i] > 0) {
66             nodes[count++] = createNode((char)i, freq[i]);
67         }
68     }
69     int n = count;
70     while(n > 1) {
71         sort(nodes, n);
72
73         struct Node* left = nodes[0];
74         struct Node* right = nodes[1];
75
76         struct Node* newNode =
77             createNode('$', left->freq + right->freq);
78         newNode->left = left;
79         newNode->right = right;
80     }

```

```

80
81     nodes[0] = newNode;
82     nodes[1] = nodes[n - 1];
83     n--;
84 }
85 struct Node* root = nodes[0];
86 int code[MAX];
87 int weightedSum = 0;
88 printf("\nHuffman Codes:\n");
89 printCodes(root, code, 0, &weightedSum);
90 float avgLength = (float)weightedSum / totalFreq;
91 printf("\nTotal Frequency = %d\n", totalFreq);
92 printf("Average Code Length = %.3f bits\n", avgLength);
93 return 0;
94 }

```

Output:

```

PS D:\raahithya\4TH SEM\DAA\week7> gcc huffman.c -o results
PS D:\raahithya\4TH SEM\DAA\week7> ./results
Enter a sentence: DATA ANALYTICS AND INTELLIGENCE LABORATORY

Huffman Codes:
O : 0000 (Length = 4)
R : 0001 (Length = 4)
C : 0010 (Length = 4)
D : 0011 (Length = 4)
L : 010 (Length = 3)
N : 011 (Length = 3)
T : 100 (Length = 3)
Y : 1010 (Length = 4)
S : 10110 (Length = 5)
B : 101110 (Length = 6)
G : 101111 (Length = 6)
E : 1100 (Length = 4)
I : 1101 (Length = 4)
A : 111 (Length = 3)

Total Frequency = 38
Average Code Length = 3.632 bits

```

Time Complexity:

The algorithm repeatedly sorts the nodes in ascending order and merges the two smallest nodes.

Since Bubble Sort is used inside a loop, sorting is done multiple times.

- Best / Average Case = $O(n^3)$
- Worst Case = $O(n^3)$

Space Complexity:

Space is required for storing the Huffman tree and node list.

Recursion is used to generate codes.

- Average Case = $O(n)$
- Worst Case = $O(n)$